

■ 4.2.7 LISP

The main similarity between *Mathematica* and LISP is that both are fundamentally symbolic languages. *Mathematica* is, however, in most respects a much higher level language than LISP.

Mathematica expressions are in many respects like LISP lists. In *Mathematica*, however, expressions are the lowest-level objects accessible to the user. LISP allows you to go below lists, and access the binary trees from which they are built. *Mathematica* hides such implementation details (in fact, *Mathematica* expressions are effectively implemented as vectors). In LISP, there are also functions like `eq`, `rplaca` and `copy` which allow you to test and modify the actual representations of objects in computer memory. *Mathematica* has no such low-level functions. All manipulation of *Mathematica* expressions is done without specific reference to their actual representation and location in computer memory. In this way, *Mathematica* can exert more control over internal details. For example, the fact that *Mathematica* does not allow user-level destructive functions means that meaningful circular internal data structures cannot be created, so that *Mathematica* can safely use more efficient reference count memory management schemes.

One fundamental difference between LISP and *Mathematica* is the way in which evaluation is done. LISP is essentially a one-step evaluation system; *Mathematica* is an infinite evaluation system. In both systems, you can define values for symbols and functions. In LISP, each evaluation (or call to `eval`) replaces any symbol or function in an expression by the value you have defined, but does not go on to perform any further evaluation on this value. In *Mathematica*, on the other hand, the evaluation goes on until the results you get no longer change.

In LISP, every object that you use must have a value defined, unless it is explicitly “quoted”. In *Mathematica*, objects are simply left unevaluated if no values are known for them.

The *Mathematica* object `Function` is a direct analog of `lambda` in LISP. In *Mathematica*, as in LISP, you can define pure functions which can be applied to expressions. For most purposes, however, it is better to define functions in *Mathematica* using transformation rules, rather than by directly creating pure functions. There are some differences between the way function arguments are treated in *Mathematica* and in LISP. When you make a call to a function in *Mathematica*, all occurrences of the function arguments that appear in the body of the function are *replaced* by the values that you give for the arguments. In LISP, on the other hand, the values of function arguments are effectively inserted by making temporary assignments, and then evaluating the body of the function.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.